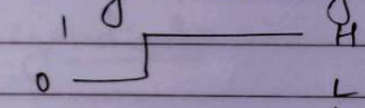
Digital systems

- Digital systems is one where both the input and the output are digital signals.
- It is widely used in.
  - Operate in one of the 2 states ON/OFF, resulting in simple circuit operation.
  - Can be realized as integrated circuits which are highly reliable, extremely small in size and cost very less.

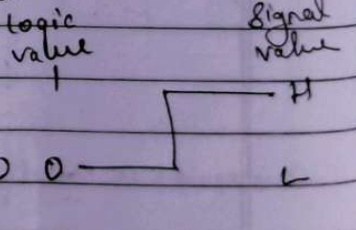
Digital circuits

- Involves circuit that has 2 possible states i.e., Binary system.
- The operation of the circuit can be described in terms of its voltage level.
  - two voltage levels High or low.

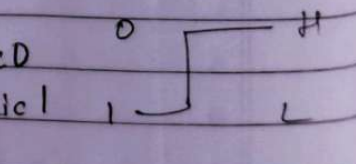


- Digital circuits can be of two types.

- Positive logic
  - Choose the high-level H as logic 1
  - Choose the low-level L as logic 0



- Negative logic
  - Choose the high-level H as logic 0
  - Choose the low-level L as logic 1



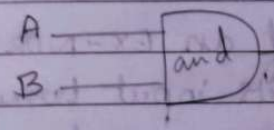
Logic Gates:

Logic gates are electronic circuits elements that performs the corresponding logical operations.

Properties of logic gates:
- logic gates handle binary signals and also generate binary signals.
- logic gates can have more than 2 inputs.

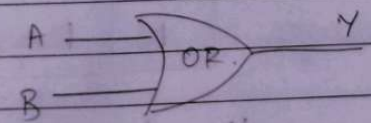
Basic Gates

AND Gate: Produces 1 at the o/p if both the inputs x and Y change to 1



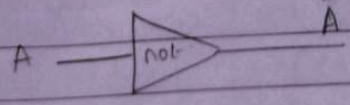
A	B	O/P(Y)
0	0	0
0	1	0
1	0	0
1	1	1

OR gate: Produces the o/p 1 if either of the logic states of inputs A and B change of 1.



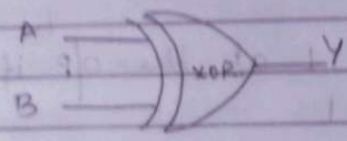
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT gate: Reverses the input and also known as inverter.



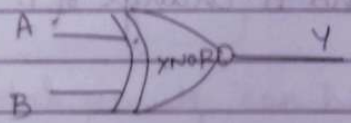
A	$\bar{A}$
0	1
1	0

EX-OR gate: The o/p of an EX-OR gate goes "HIGH" only when both of its input terminals are at "DIFFERENT" logic levels with respect to each other. It implements the function "A OR B but NOT both".



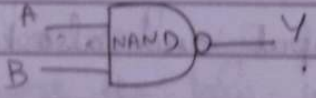
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

EX-NOR gate: The output of an EX-NOR gate goes "HIGH" when both of its input terminals are at "SAME" logic levels with respect to each other and is also known as equivalence gate.



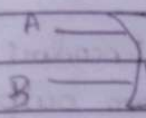
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

NAND gate: It has the opposite o/p to the AND gate. It is NOT-AND.



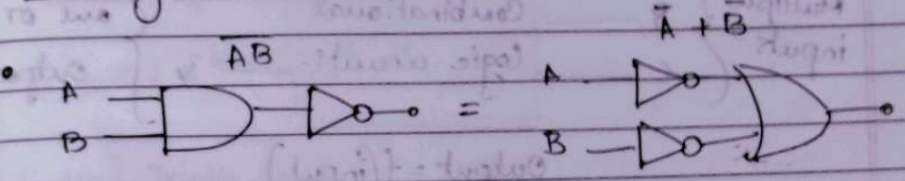
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR gate: It has the opposite o/p to the OR gate. It is NOT-OR.

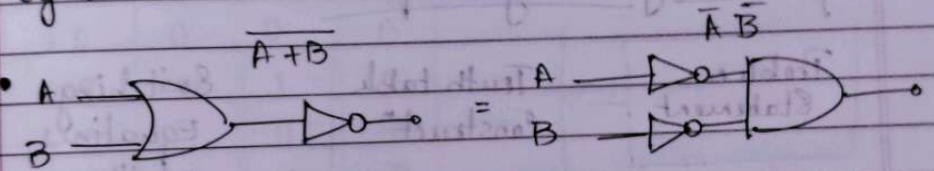


A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

De Morgan's Theorem



A NAND gate is equivalent to an inversion followed by an OR.



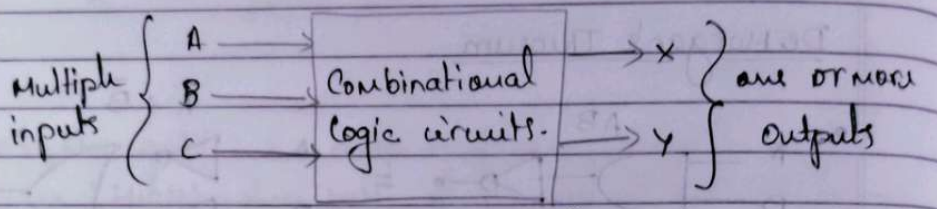
A NOR gate is equivalent to an inversion followed by an AND.

Boolean Laws

Law / Theorem	Law of Addition	Law of Multiplication
Identity law	$X+0=X$	$X\cdot 1=X$
Complement law	$X+X'=1$	$X\cdot X'=0$
Idempotent law	$X+X=X$	$X\cdot X=X$
Dominant law	$X+1=1$	$X\cdot 0=0$
Involution law	$(X')'=X$	
Commutative law	$X+Y=Y+X$	$X\cdot Y=Y\cdot X$
Associative law	$X+(Y+Z)=(X+Y)+Z$	$X\cdot(Y\cdot Z)=(X\cdot Y)\cdot Z$
Distributive law	$X\cdot(Y+Z)=X\cdot Y+X\cdot Z$	$X+Y\cdot Z=(X+Y)\cdot(X+Z)$
Demorgan's law	$(X+Y)'=X'\cdot Y'$	$(X\cdot Y)'=X'+Y'$
Absorption law	$X+(X\cdot Y)=X$	$(X\cdot(X+Y)=X$

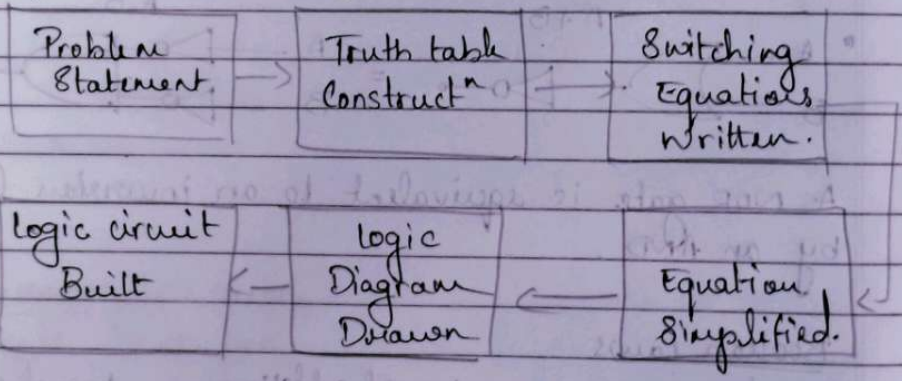
Definition of Combinational logic

- Combinational logic deals with the techniques of "combining" the basic gates into circuits that perform some desired function.
  eg: Adders, Subtractors, Decoders, Encoders, Multiplexers
- logic circuits without feedback from output to input.
- logic circuits that contain no memory.



Output = f(input)

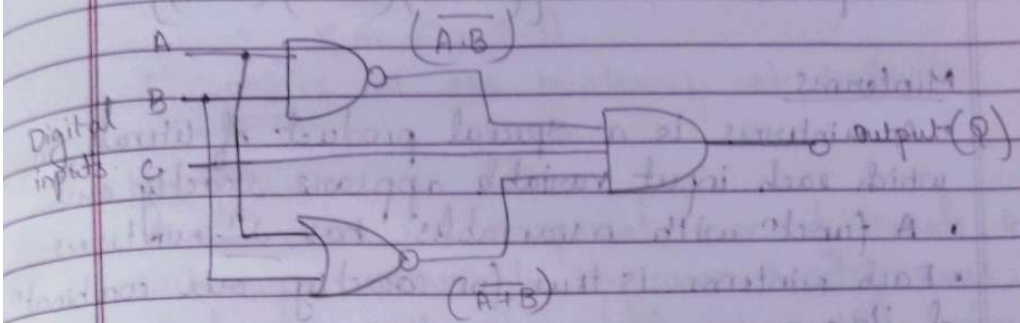
General logic Design sequence:



Combinational logic Specification.

1] Logic Diagram: graphical representation.
2] Truth table: A list that shows all the o/p states in tabular form for each possible combination of input variable.
3] Boolean Equations: Algebric expressions showing the operation of the logic circuit for each input variable.

Logic Gates & Logic Diagram



Boolean Expression:

$$Q = (\overline{A \cdot B}) \cdot (\overline{A+B}) \cdot C$$

Typical Truth table:

C	B	A	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Boolean Equations or Switching equations:

- A literal is a Boolean variable or its complement (a, x', y)
- A product-term is a literal or the logical product (AND) of multiple literals (ab, x'y, z'y')
- A Sum term is a literal or the logical sum (OR) of multiple literals (a+b, a'+b, x'+y')
- A sum of product (SOP) is the logic OR of multiple product terms (ab+bc'+a'b')

- A product of sum (POS) is the logical AND of multiple sum terms $[(a+b')(a'+b')(a'+b)]$

Minterms

- A minterms is a special product of literals, in which each input variable appears exactly once.
- A funct^n with n variables has 2^n minterms.
- Each minterm is true for exactly one combinat^n of i/p s

Minterm	Is true when ...	Shorthand
$x'y'z'$	$x=0; y=0; z=0$	M_0
$x'y'z$	$x=0; y=0; z=1$	M_1
$x'yz'$	$x=0; y=1; z=0$	M_2
$x'yz$	$x=0; y=1; z=1$	M_3
$xy'z'$	$x=1; y=0; z=0$	M_4
$xy'z$	$x=1; y=0; z=1$	m_5
xyz'	$x=1; y=1; z=0$	m_6
xyz	$x=1; y=1; z=1$	m_7.

Sum of Minterms

x	y	z	f(x,y,z)	f'(x,y,z)
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	0	1

$$f = x'y'z' + x'y'z + x'yz' + x'yz + xyz'$$
$$= m_0 + m_1 + m_2 + m_3 + m_6$$
$$= \Sigma m(0,1,2,3,6)$$

$f' = xy'z' + xy'z + xyz$

$= m_4 + m_5 + m_7$

$= \Sigma m(4,5,7)$

f' contains all the minterms not in f.

Maxterms

- A maxterm is a special sum of literals, in which each input variable appears exactly once.
- A funct^n with n variables has 2^n maxterms.
- Each maxterm is false for exactly one combinat^n of inputs.

Maxterm.	Is false when...	Shorthand
$x+y+z$	$x=0; y=0; z=0$	M_0
$x+y+z'$	$x=0; y=0; z=1$	M_1
$x+y'+z$	$x=0; y=1; z=0$	M_2
$x+y'+z'$	$x=0; y=1; z=1$	M_3
$x'+y+z$	$x=1; y=0; z=0$	M_4
$x'+y+z'$	$x=1; y=0; z=1$	M_5
$x'+y'+z$	$x=1; y=1; z=0$	M_6
$x'+y'+z'$	$x=1; y=1; z=1$	M_7

Product of Maxterms.

x	y	z	f(x,y,z)	f'(x,y,z)
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	0	1

$f = (x'+y+z)(x'+y+z')(x'+y'+z')$
$= M_4 M_5 M_7$
$= \pi M(4,5,7)$

$f' = (x+y+z)(x+y+z')(x+y'+z)(x+y'+z')$
$(x'+y'+z)$
$= M_0 M_1 M_2 M_3 M_6$
$= \pi M(0,1,2,3,6)$

f' contains all the maxterms not in f.

Relation of Minterm to Maxterm.

- Any minterm m_i is the complement of the corresponding maxterm M_i

Minterm	Shorthand	Maxterm	Shorthand
$x'y'z'$	m_0	$x+y+z$	M_0
$x'y'z$	m_1	$x+y+z'$	M_1
$x'yz'$	m_2	$x+y'+z$	M_2
$x'yz$	m_3	$x+y'+z'$	M_3
$xy'z'$	m_4	$x'+y+z$	M_4
$xy'z$	m_5	$x'+y+z'$	M_5
xyz'	m_6	$x'+y'+z'$	M_6
xyz	m_7	$x'+y'+z'$	M_7

Canonical Forms.

- A canonical form of a switching equation is one that contains all of the available input variables.

> Canonical Sum of Products: It is a complete set of Minterms that defines when an output is a logical 1

> Canonical Product of sums: It is a complete set of maxterms that defines when an output is a logical 0.

- Canonical expressions are not simplified and contains redundancies.

f: $x'y' + x'y + xyz'$ is a normal expression.
$x'y'z' + x'y'z + x'yz' + x'yz + xyz'$ is the canonical form.

Conversion to canonical form:

- SOP to canonical:
  - Identify the missing variable in each AND term.
  - AND the missing term and its complement with the original AND term.
  - Expand the term by distribution property.

- POS to canonical:
  - Identify the missing variable in each OR term.
  - OR the missing term and its complement with the original OR term.
  - Expand the term by distribution property.

Grouping in KMap:

Rules of grouping:

- 1's & 0's can not be grouped
- diagonal 1's can not be grouped.
- Elements in a grp should be 2^n.
- Minimum grp should be formed.
- Overlapping is applicable.

The three variable KMap.

c \ ab	a'b' 00	a'b 01	ab 11	ab' 10
c'0	000 0	010 2	110 6	100 4
c 1	001 1	011 3	111 7	101 5

Dont care Conditions.

- In some cases, the function is not specified for certain combinat^n of input variables as 1 or 0.
- There are 2 cases in which it occurs.
  - The input combination never comes.
  - The input combination occurs but we do not care what the outputs are in response to these inputs because the output will not be observed.
- In both cases, the outputs are called as unspecified and the functions having them are called as incompletely specified functions.
- In most applicat^{n}s, we simply do not care what value is assumed by the functions for unspecified minterms.
- A don't care conditions, marked by (x) in the truth ~~BCD to Excess 3 code converter (1/2)~~ table, indicates a condition where the design doesn't care if the output is a (0) or (1)
- A don't care condition can be treated as a (0) or a (1) in a k-Map.
- Treating a don't care as a (0) means that you do not need to group it.
- Treating a don't care as a (1) allows you to make a grouping larger, resulting in a simplified term in the SOP equation.

BCD to Excess 3 code Converter (½)

Truth table:

	BCD				Excess-3			
	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0
10	1	0	1	0	x	x	x	x
11	1	0	1	1	x	x	x	x
12	1	1	0	0	x	x	x	x
13	1	1	0	1	x	x	x	x
14	1	1	1	0	x	x	x	x
15	1	1	1	1	x	x	x	x

K Map:

W

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	x	x	x	x
10	1	1	x	x

B; D, C

X

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	0	0	0
11	x	x	x	x
10	0	1	x	x

B; A; D, C

Y

AB \ CD	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	x	x	x	x
10	1	0	x	x

B; D

Z

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	x	x	x	x
10	1	0	x	x

B; D

$W = A + BC + BD = A + B(C+D)$

$X = B'C + B'D + BC'D =$

$= B'(C+D) + BC'D'$

$Y = CD + C'D'$

$Z = D'$

Logic Diagram